

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document provides a detailed description of the WeatherGuard system, including its functionalities, system architecture, constraints, and design considerations. It serves as a reference for developers, project supervisors, and stakeholders to ensure proper understanding and implementation of the system.

1.2 Scope

The WeatherGuard system is an intelligent weather monitoring and disaster alert platform that provides real-time weather updates, predictive risk analysis, and location-based alerts. The system enables users to monitor environmental conditions, assess potential risks, and access emergency preparedness guidelines.

The system includes features such as user management, weather data retrieval, risk assessment using predefined models, alert generation, and emergency guidance. It is designed as a scalable web-based application.

However, the system is limited to software-based predictions and depends on external APIs for weather data. Future enhancements may include AI-based forecasting and IoT integration.

1.3 Definitions, Acronyms, Abbreviations

- SRS – Software Requirements Specification
- API – Application Programming Interface
- UI – User Interface
- DBMS – Database Management System
- GPS – Global Positioning System

1.4 Overview

This document outlines the functional and non-functional requirements of the WeatherGuard system. It also describes system constraints, assumptions, and dependencies to ensure a structured development approach.

2. PROBLEM STATEMENT

Existing weather applications mainly provide general weather forecasts but lack integrated disaster alerts, risk assessment, and emergency guidance. Users rely on multiple sources,

causing delays and confusion during emergencies. A unified system is required to provide real-time information and safety support.

3. OBJECTIVES

- To develop a system that provides real-time weather updates
- To integrate disaster alert mechanisms
- To implement a Smart Risk Indicator
- To provide location-based personalized services
- To include an Emergency Preparedness module

4. LITERATURE REVIEW

4.1 Existing System

Current systems include weather apps and government platforms.

Features:

- Display weather conditions (temperature, humidity, forecasts)
- Provide limited notifications

Limitations:

- No integration of disaster alerts
- Lack of risk prediction mechanisms
- No emergency guidance
- Fragmented information sources
- Poor decision-making support

4.2 Proposed System

The proposed system (WeatherGuard) overcomes these limitations.

Features:

- Real-time weather data via APIs
- Integrated disaster alerts

- Smart Risk Indicator (Low, Medium, High)
- Emergency preparedness guidance
- User-friendly dashboard

Advantages:

- Centralized platform
- Faster response during emergencies
- Improved safety awareness
- Better user experience

System Features (Functional Requirements)

- **3.1 User Management**

- The system shall allow users to register and log in securely.
- The system shall allow users to update profile and location preferences.

- **3.2 Weather Data Retrieval**

- The system shall fetch real-time weather data using external APIs.
- The system shall display temperature, humidity, rainfall, and wind speed.

- **3.3 Risk Analysis**

- The system shall utilize an existing RiskAnalysisService to compute risk levels.
- The system shall analyze multiple parameters such as temperature, rainfall, humidity, and wind speed.
- The system shall classify risk levels as Low, Medium, or High.

- **3.4 Alerts System**

- The system shall generate alerts for severe weather conditions.
- The system shall support a hybrid alert mechanism (API + simulated alerts).
- The system shall categorize alerts based on severity.

3.5 Emergency Preparedness

- The system shall provide disaster-specific safety guidelines.
- The system shall display recommendations based on risk levels.

3.6 Data Visualization

- The system shall present weather trends using charts and graphs.

3.7 Admin Functions

- The admin shall manage alerts and monitor system activity.

4. External Interface Requirements

4.1 User Interface

- Responsive web interface
- Simple navigation and dashboard view

4.2 Hardware Interface

- Standard computing devices (PC, mobile)

4.3 Software Interface

- Weather APIs
- Database system

4.4 Communication Interface

- HTTP/HTTPS protocols

5. Non-Functional Requirements

5.1 Performance

- The system shall provide fast response times for real-time data.

5.2 Reliability

- The system shall ensure high availability and fault tolerance.

5.3 Security

- The system shall implement secure authentication and data protection.

5.4 Usability

- The system shall provide an intuitive user interface.

5.5 Scalability

- The system shall support increasing users and data load.

5.6 Maintainability

- The system shall be modular and easy to update.

6. Database Design (Aligned with System)

Entities:

- User
- Location
- WeatherData
- RiskData
- Alert
- EmergencyGuidelines
- **Key Tables:**

WeatherData

- weather_id, location_id, temperature, humidity, rainfall, wind_speed, timestamp

RiskData

- risk_id, weather_id, risk_level, temperature, rainfall, wind_speed, humidity, description, timestamp

Alert

- alert_id, location_id, alert_type, severity, message, source, timestamp
- **EmergencyGuidelines**
- guideline_id, risk_level, disaster_type, description

7. System Workflow

- Weather data is fetched from external APIs
- Data is stored in the database

- RiskAnalysisService evaluates risk
- Risk level is stored and displayed
- Alerts are generated
- Emergency guidelines are provided
- shall be encrypted and protected against unauthorized access.

6.5 Scalability

- The system shall support future expansion with increased users and data load.

6.6 Maintainability

- The system shall be modular and easy to update or modify.
- Proper documentation shall be maintained for future enhancements.

7. SOFTWARE AND HARDWARE REQUIREMENTS

Software

- Frontend: React
- Backend: Spring Boot
- Database: MySQL
- Tools: Postman, VS Code

Hardware

- Minimum 4GB RAM
- Internet connection

8. USER ROLES

Admin

- Manage system data
- Monitor alerts

User

- View weather updates

- Check alerts and risk levels
- Access emergency information

9. ASSUMPTIONS AND CONSTRAINTS

Assumptions

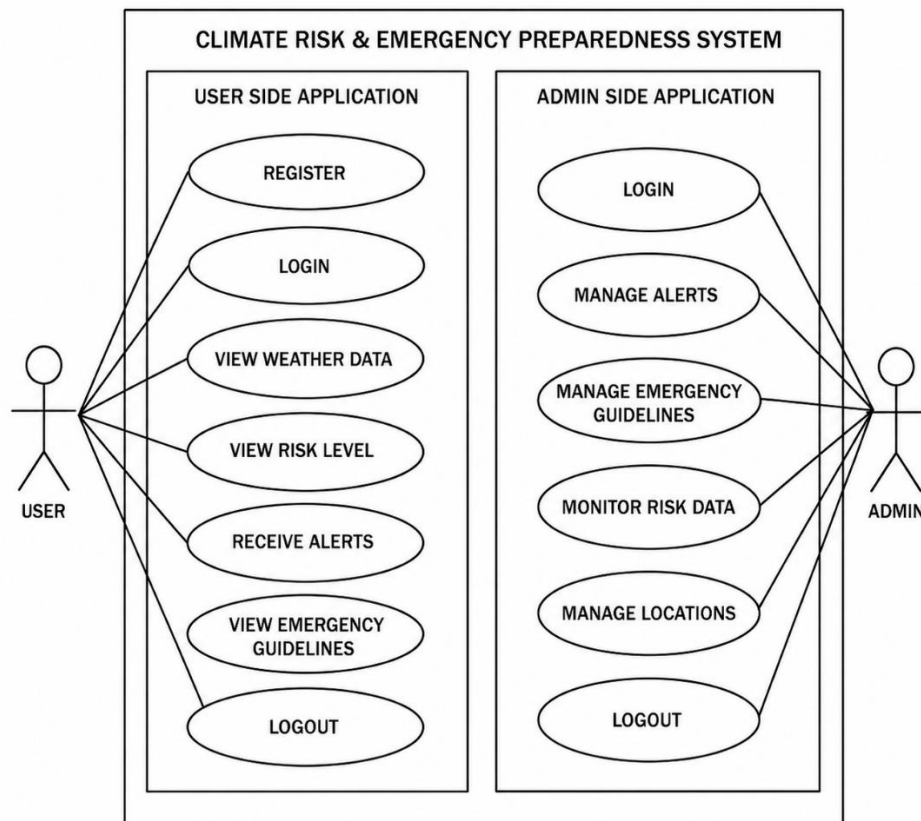
- Users have internet access
- APIs are available

Constraints

- Dependence on third-party APIs
- Limited real-time government data access

10. SYSTEM DESIGN DOCUMENT (SDD)

10.1 Use Case Diagram (Description)



Actors:

- User
- Admin

User Side Use Cases

- Register
- Login
- View Weather Data
- View Risk Level
- Receive Alerts
- View Emergency Guidelines
- Logout

Admin Side Use Case

- Login
- Manage Alerts
- Manage Emergency Guidelines
- Monitor Risk Data
- Logout

10.2 DATABASE DESIGN

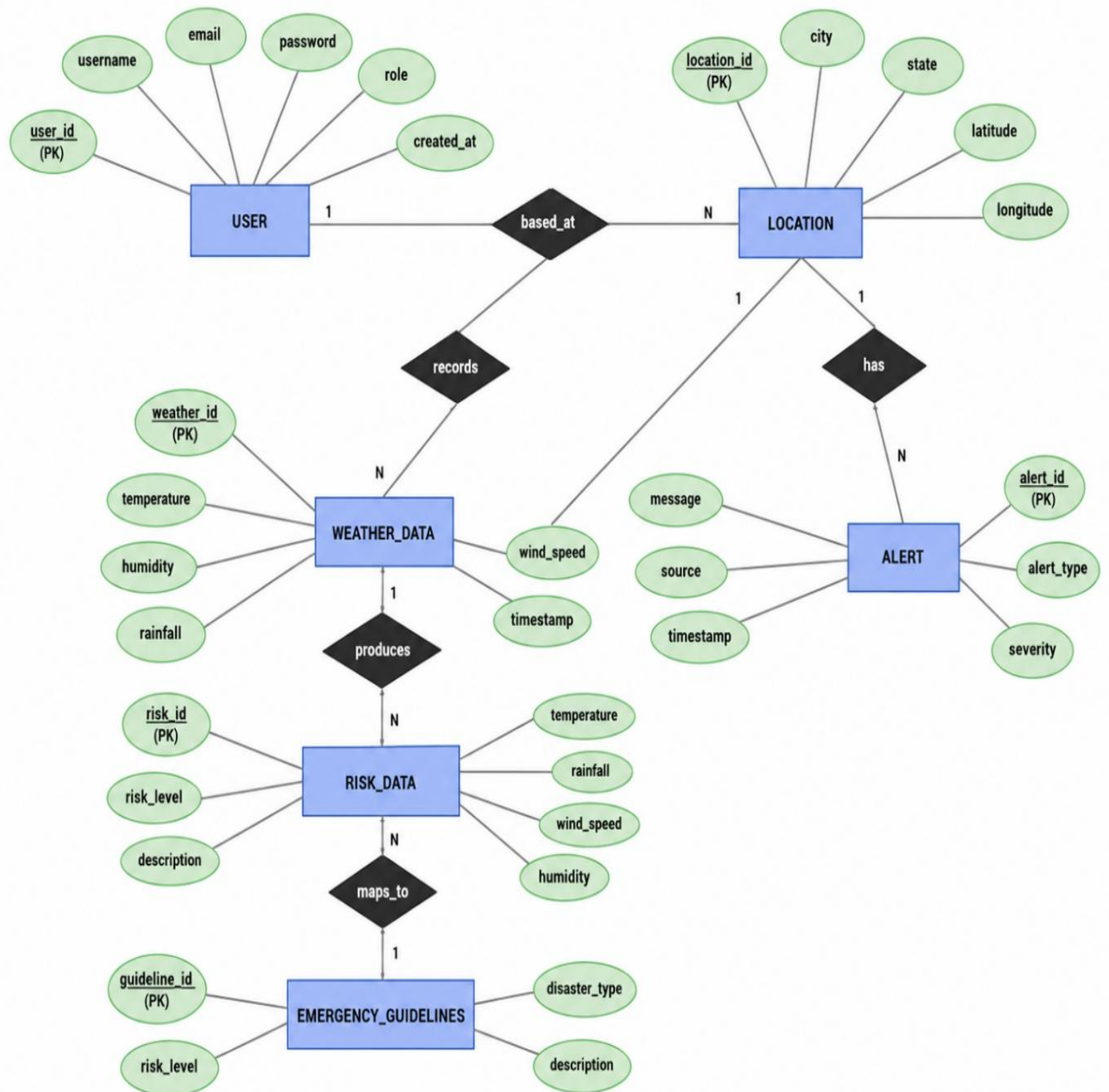
10.2.1 ER Diagram

Entities:

- Users
- WeatherData
- Alerts
- RiskData

Relationships:

- Users access weather data
- Alerts linked to location
- Risk derived from weather data



10.2.2 TABLES

Table 1: Users

Field Name	Data Type / Constraint	Description
user_id	INT, Primary Key, Auto Increment	Unique identifier for each user
username	VARCHAR, UNIQUE, NOT NULL	Username of the user
email	VARCHAR, UNIQUE, NOT NULL	Email address of the user
password_hash	VARCHAR, NOT NULL	Encrypted password for authentication
role	ENUM ('user', 'admin')	Defines user role (normal user or admin)
location_id	INT, Foreign Key	References Location(location_id)
created_at	TIMESTAMP	Timestamp of account creation

Table 2: Location

Field Name	Data Type / Constraint	Description
location_id	INT, Primary Key, Auto Increment	Unique identifier for location
city	VARCHAR	Name of the city
state	VARCHAR	Name of the state
latitude	DECIMAL	Latitude coordinate
longitude	DECIMAL	Longitude coordinate

Table 3: Weather Data

Field Name	Data Type / Constraint	Description
weather_id	INT, Primary Key, Auto Increment	Unique identifier for weather record
location_id	INT, Foreign Key	References Location(location_id)

temperature	DECIMAL	Temperature value
humidity	DECIMAL	Humidity level
rainfall	DECIMAL	Rainfall measurement
wind_speed	DECIMAL	Wind speed value
timestamp	TIMESTAMP	Time of data recording

Table 4: RiskData

Field Name	Data Type / Constraint	Description
risk_id	INT, Primary Key, Auto Increment	Unique identifier for risk record
weather_id	INT, Foreign Key	References WeatherData(weather_id)
risk_level	ENUM ('Low', 'Medium', 'High')	Classified risk level
temperature	DECIMAL	Temperature used for risk calculation
rainfall	DECIMAL	Rainfall used for risk calculation
wind_speed	DECIMAL	Wind speed used for risk calculation
humidity	DECIMAL	Humidity used for risk calculation
description	VARCHAR	Explanation of risk level
timestamp	TIMESTAMP	Time of risk evaluation

Table 5: Emergency Guidelines

Field Name	Data Type / Constraint	Description
guideline_id	INT, Primary Key, Auto Increment	Unique identifier for each guideline
risk_level	ENUM ('Low', 'Medium', 'High')	Associated risk level
disaster_type	VARCHAR	Type of disaster (e.g., Flood, Heatwave, Storm)

description	TEXT	Detailed emergency instructions
timestamp	TIMESTAMP	Time of guideline creation/update